CS-354: Complier Construction

(Spring 2026)

Lecture 1

Adnan Rashid

Email: adnan.rashid@seecs.edu.pk

Web: https://adrashid.github.io/

Outline

- Course Objectives
- Introduction
  - Compilers
  - About us
- Assessment Plan
- General Instructions

Lecture's Content

Compiler: Principles, Techniques and Tools (2nd Edition)

by

Alfred V. Aho et al.

Section 1.1 (Page 1 onwards)

Objectives of Course

Objective 1: Understand compilers and languages

- Understand the code structure
- Understand language semantics
- Understand the relationship between the source code and the generated machine code
- Become a better programmer

Objectives of Course

Objective 2: Nice balance of theory and practice

- Theory
  - Mathematical models: Regular expressions, automata, grammars, graphs
  - Algorithms that use these models
- Practice
  - Apply theoretical notions to build a real compiler and/or its components

Objectives of Course

Objective 3: Programing experience

- Write a large program which manipulates complex data structures
- Learn more about C/C++/Java and Intel x86 etc.

What are Compilers?

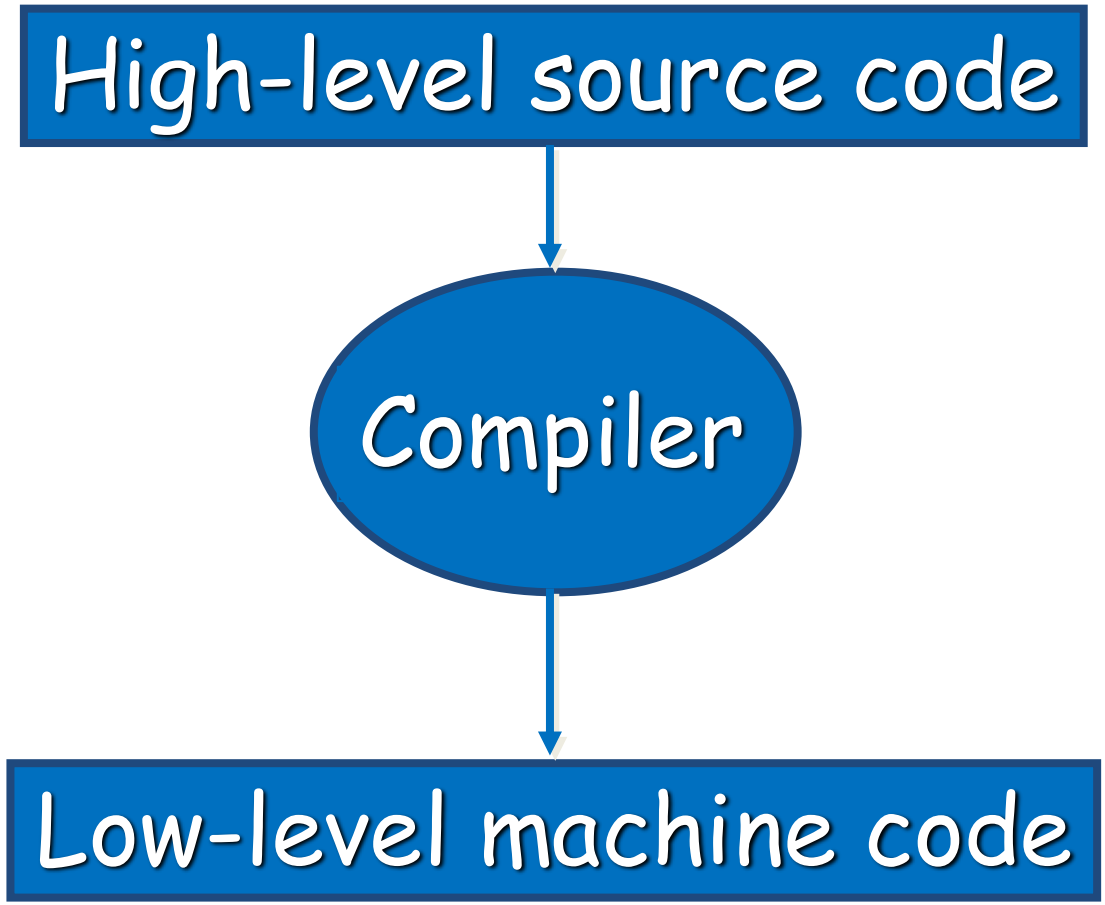
- Programming languages are notations for describing computations to people and to machines
  - High Level Languages (Java, C/C++, Python, …)
  - Low Level Languages (Assembly, ByteCode, …)
  - Machine Language (00110011, 11101100, …)
- Machines do not understand programming languages!
  - Therefore, a software system is required to do the translation
  - This is the compiler

He always liked to open the Annual Programmer's Conference with a joke in binary.

What are Compilers?

- Translate information from one representation to another; usually information = program
- A program that reads a program written in one language (source language) and translates it to an equivalent program in another language (target language) -- Aho et al
- Compilers are also known as Language Processors
- Typical Compilers:
  - VC, VC++, JavaC, Clang, FORTRAN, Pascal, …
- Translators
  - Word to PDF
  - PDF to Postscript

Typical Compilation



High-level Source Code

```
int expr( int n )
{
 int d;
 d = 4*n*n*(n+1)*(n+1);
 return d;
}
```

- Optimized for human readability
- Matches human notions of grammar
- Uses named constructs such as variables and procedures

Low-level Source Code

```
.globl _expr
_expr:
  pushl %ebp
  movl %esp,%ebp
  subl $24,%esp
  movl 8(%ebp),%eax
  movl %eax,%edx
  leal 0(,%edx,4),%eax
  movl %eax,%edx
  imull 8(%ebp),%edx
  movl 8(%ebp),%eax
  incl %eax
```

```
  imull %eax,%edx
  movl 8(%ebp),%eax
  incl %eax
  imull %eax,%edx
  movl %edx,-4(%ebp)
  movl -4(%ebp),%edx
  movl %edx,%eax
  jmp L2
  .align 4
L2:
  leave
  ret
```

Low-level Source Code

- Optimized for hardware
- Consists of machine instructions
- Uses registers and unnamed memory locations
- Much harder to understand by humans

```
.globl _expr
_expr:
  pushl %ebp
  movl %esp,%ebp
  subl $24,%esp
  movl 8(%ebp),%eax
  movl %eax,%edx
  leal 0(,%edx,4),%eax
  movl %eax,%edx
  imull 8(%ebp),%edx
  movl 8(%ebp),%eax
  incl %eax
```

```
  imull %eax,%edx
  movl 8(%ebp),%eax
  incl %eax
  imull %eax,%edx
  movl %edx,-4(%ebp)
  movl -4(%ebp),%edx
  movl %edx,%eax
  jmp L2
  .align 4
L2:
  leave
  ret
```

How to Translate?

- Correctness
  - The generated machine code must execute precisely the same computation as the source code
- Is there a unique translation?
  - No!
- Is there an algorithm for an ideal translation?
  - No!

Ideal: time, memory, power trade off

How to Translate?

- Translation is a complex process
- Source language and generated code are very different
- Need to structure the translation process
  - More about structure of compilers: Later!

About Us

- Instructor
  - Dr. Adnan Rashid (B-201, First Floor, IAEC)
    - Assistant Professor, NUST (October 2020 – June 2023)
    - Postdoctoral Fellow, Concordia University (July 2023 – December 2025)
    - Assistant Professor, NUST (January 2026 – Present)
  - Contact: adnan.rashid@seecs.edu.pk
  - Web: https://adrashid.github.io/
    https://seecs.nust.edu.pk/faculty/adnan-rashid/

About Us

- Updates (on LMS, Email):

- Updates and Discussions (on WhatsApp):
  https://tinyurl.com/2taw3bv7

About Us

- Code for Course Self Enrollment on LMS: 960832451
- Office Hours: By appointment only (via email)

Class Schedule, LMS and CMS

Day	BSCS-12A
Wednesday	0900-0950 (CR-07-UG Block)
Thursday	1400-1450 (CR-07-UG Block)
Friday	1000-1050 (CR-07-UG Block)

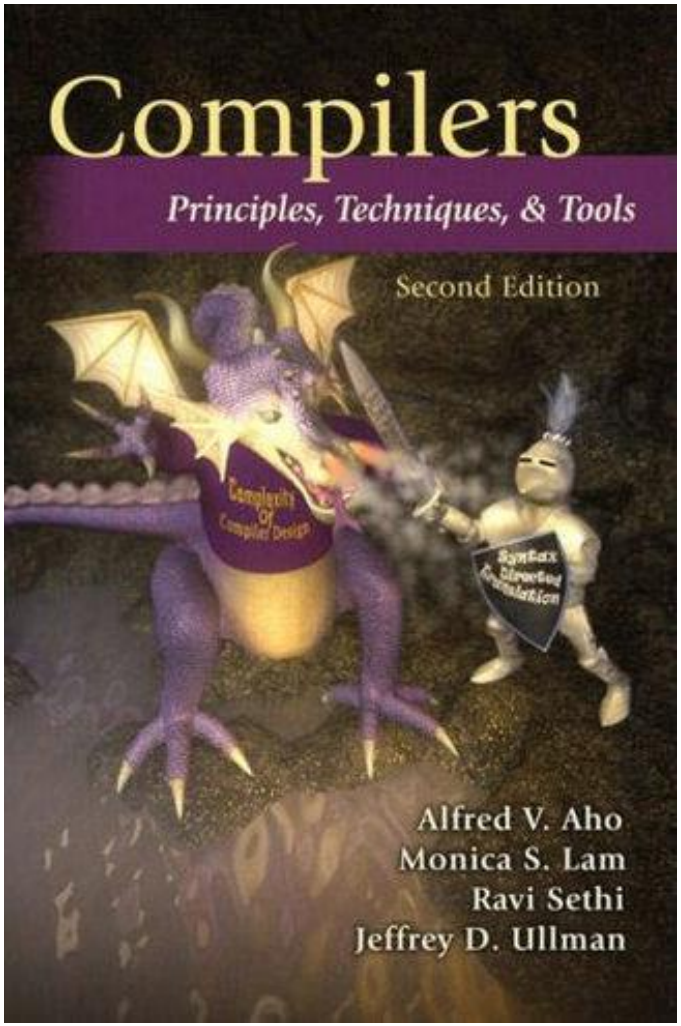
- Consult LMS for
  - Lecture Notes, Quizzes and Assignments
- Consult Qalam for
  - Attendance and grades
  - Online Feedback

Attendance Policy

- 75-100% required at the end of semester!
- Attendance will be taken at any moment
- No compensation for attendance!

Textbook

- Alfred V. Aho et al., Compilers: Principles, Techniques and Tools (Second Edition), 2/E, Pearson, 2007 also known as the "Dragon Book"



Assessment Plan (Tentative)

- 40% Final Exam (Closed Books, Closed Notes, Comprehensive)
- 30% Mid Term (Closed Books, Closed Notes, Comprehensive)
- 10% Quizzes (Minimum 05, Surprise!)
- 10% Project
- 10% Assignments (Minimum 05)

Homework/Assignment Policy

- All homework assignments must be done individually or as directed
  - Require uploading a soft-copy on LMS (for plagiarism check)
- Cheating
  - Helping others, getting help, looking up websites for solutions etc.
- Late Submissions
  - Late submission will get a -20% penalty up to one day
  - -40% penalty up to 2 days (No acceptation after 2 days from deadline)

Homework/Assignment Policy

- Contact instructor at least 03 days before the deadline in case of emergency

- Any deviation from the above rule will be considered cheating and will be subject to the SEECS academic dishonesty policy

  - See more details in UG Handbook

Lecture Basics

- Classes will mainly involve Slides
  - Lectures (pdf) will be available on LMS
  - However, no scribes from the class will be made available
  - Therefore, take your own notes in the class

- For latest/updated slides, download before each use
  - As I might update/correct slides at any stage

Thank you!